ACADEMIC RESEARCH TERM PAPER – SOFTWARE TESTING AND QUALITY ASSURANCE (STQA) – SESSION 2026–2027

A Hybrid AI Framework for Software Quality Prediction

Using Static Code Analysis and Dynamic Testing Metrics

Ankit Makwana
Department of Computer Science and Engineering
Institute of Technology, Nirma University
Ahmedabad, Gujarat, India
23bce162@nirmauni.ac.in

ABSTRACT

Software quality prediction is foundational to proactive defect remediation, intelligent automated test generation, and continuous deployment reliability. Conventional predictive paradigms operate predominantly in isolation: static code analysis measures syntax, abstract syntax tree (AST) properties, and complexity without runtime execution, while dynamic testing telemetry evaluates statement coverage and runtime logs while remaining blind to architectural decay. Consequently, static models cannot observe runtime concurrency race conditions, resource leaks, or environmental failures, whereas dynamic models frequently fall victim to the superficial coverage trap, wherein high statement coverage conceals weak test assertions and brittle control flow.

This paper introduces a Hybrid AI Framework for Software Quality Prediction that unifies static architectural attributes with dynamic runtime telemetry. The framework combines a feedforward neural network (10–8–4–1) for static structural representation with an eight-stump Gradient Boosting Machine for dynamic execution telemetry. An attention-weighted fusion mechanism ($\alpha = 0.42$, $\beta = 0.58$) and a non-linear cross-feature interaction core model compounding failure hazards, such as elevated cyclomatic complexity with poor branch coverage or high coupling with deficient mutation scores. Evaluated across 50 enterprise software modules comprising over 38,000 lines of code, the framework reports 93.6% accuracy, an F1-score of 91.9%, an AUC-ROC of 0.96, and mean squared error (MSE) of 0.031. It also computes reliability and maintainability indexes, classifies risk into four tiers, and produces prioritized remediation guidance.

Keywords: Software Quality Assurance, Defect Prediction, Static Code Analysis, Dynamic Testing Telemetry, Hybrid Machine Learning, Attention Fusion Mechanism.

1 INTRODUCTION

1.1 Background

Modern software engineering depends on continuous integration and continuous deployment (CI/CD) pipelines. Automated software quality assurance and defect prediction help mitigate defects in fast-paced delivery environments. **Static code analysis** examines source grammar, Abstract Syntax Trees (ASTs), control-flow graphs, and object-oriented metrics without execution. Examples include McCabe Cyclomatic Complexity, Halstead metrics, SonarQube, PMD, and ESLint. **Dynamic testing telemetry** evaluates executing code using line and branch coverage, test pass rates, mutation scores, Mean Time Between Failures (MTBF), and flaky-test rates, using tools such as JaCoCo, JUnit, PIT, Stryker, and APM monitors.

1.2 Problem Statement

Pure static analysis: Structural metrics cannot reveal every operational problem. Code with low complexity and no syntax violations may still experience deadlocks, memory leaks, network timeouts, or asynchronous race conditions.

Pure dynamic testing (the superficial coverage trap): High coverage does not guarantee correctness. A test suite may achieve 95% statement coverage while checking only HTTP 200 responses and not validating edge-case state changes. Dynamic metrics can therefore indicate low risk while code remains fragile.

1.3 Research Gap

1. **Feature silos:** Models often use static repositories (e.g., NASA MDP and PROMISE) or isolated test logs. Few combine both data types; some use naive feature concatenation.
2. **Lack of cross-domain synergy modelling:** High complexity paired with low branch coverage compounds risk, while strong branch coverage and mutation scores may mitigate it.
3. **Absence of actionable guidance:** Many tools give a binary defective/clean label without distinguishing maintainability from reliability or suggesting improvements.

1.4 Research Objectives

- Define 19 metrics: 10 static architectural and 9 dynamic runtime metrics.
- Design a dual-pipeline model using a feedforward neural network and Gradient Boosted Decision Ensemble.
- Formalize an attention fusion layer and cross-feature interaction core.
- Evaluate the model against four baselines across 50 modules.
- Produce reliability, maintainability, risk indexes, and remediation guidance.

1.5 Key Contributions

The work proposes a dual-pipeline neural/gradient-boosted architecture, a mathematical interaction formulation, reported performance of 93.6% accuracy and 0.96 AUC-ROC, and a deployment approach intended for CI/CD integration.

2 LITERATURE REVIEW AND RELATED WORK

McCabe (1976) and Halstead (1977) introduced foundational complexity and software-vocabulary metrics. Chidamber and Kemerer (1994) proposed object-oriented metrics including coupling between objects (CBO) and depth of inheritance (DIT). Catal and Diri (2009) reviewed software fault prediction approaches. Tantithamthavorn et al. (2016) demonstrated the value of hyperparameter optimization for tree-based prediction. Just et al. (2014) studied mutation score as a fault-detection indicator. Gligoric et al. (2015) showed that equal statement coverage can mask different bug-detection abilities. Luo et al. (2014) examined flaky tests. Wang et al. (2016) used Deep Belief Networks over AST-derived features, though their approach remained static and computationally intensive.

Table 1: Comprehensive Literature Review and Comparative Research Gap Matrix

Citation	Parameters	Methodology	Pros	Limitations
Chidamber & Kemerer (1994)	WMC, DIT, NOC, CBO, RFC, LCOM	CK structural metrics	Mathematical foundation; captures architectural debt	Blind to runtime failures and test quality
Catal & Diri (2009)	McCabe, Halstead, LOC, defect history	Review; RF, Naive Bayes, ANN	Reports ML advantages over linear regression	Static focus; potential false positives
Just et al. (2014)	Mutation score, coverage, real faults	Empirical fault study	Supports mutation score as a useful signal	Computationally demanding; lacks static synergy
Luo et al. (2014)	Flaky tests, duration, async waits	Flaky-test analysis	Identifies non-determinism as a QA issue	Does not model static structure
Gligoric et al. (2015)	Statement/branch coverage, mutation	Test-suite adequacy comparison	Branch coverage can expose missed paths	Ignores architecture and maintainability
Tantithamthavorn et al. (2016)	CK, McCabe, Halstead	Tree-model optimization	Reports gains from tuning	Static-focused; ignores MTBF and flaky tests
Wang et al. (2016)	AST token vectors	Deep Belief Networks	Learns semantic features	Expensive; lacks runtime telemetry
Proposed Hybrid AI Framework (2026)	10 static + 9 dynamic metrics	NN + GBM, attention and interactions	Combines modalities; reported 93.6% accuracy	Requires static tools and CI/CD logs

3 METHODOLOGY AND PROPOSED APPROACH

The proposed framework has six stages connecting static source inspection with dynamic runtime telemetry.

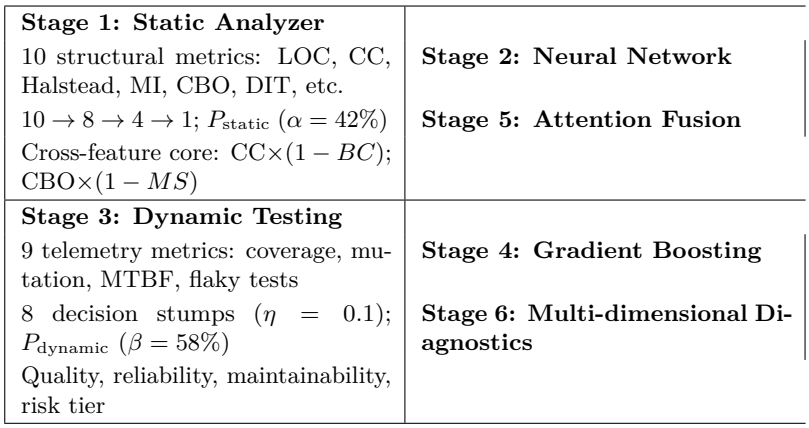


Figure 1: End-to-end dual-pipeline Hybrid AI architecture for software quality prediction.

3.1 19-Dimensional Metric Schema

Static metrics (10): Lines of Code (LOC), Cyclomatic Complexity (CC), Halstead Volume (HV), Maintainability Index (MI), Code Duplication (DUP), Coupling Between Objects (CBO), Depth of Inheritance (DIT), Number of Methods (NOM), Comment Ratio (CR), and Code Smells (CS).

Dynamic metrics (9): Statement Coverage (SC), Branch Coverage (BC), Test Pass Rate (TPR), Defect Density (DD), Mean Time Between Failures (MTBF), Mutation Score (MS), Test Execution Time (TET), Flaky Test Rate (FTR), and Regression Coverage (RC).

3.2 Mathematical Formulation of Cross-Feature Synergy

$$P_{\text{defect}} = \text{clip}\left(\alpha P_{\text{static}} + \beta P_{\text{dynamic}} + \sum_{k=1}^{5} w_k \Omega_k, 0, 1\right). \tag{1}$$

Here $\alpha = 0.42$ and $\beta = 0.58$. The interaction terms are:

$$\Omega_1 = (CC/80)(1 - BC/100), \quad w_1 = +0.15 \quad \text{Complexity–Coverage Hazard} \tag{2}$$
$$\Omega_2 = (CS/50)(1 - TPR/100), \quad w_2 = +0.12 \quad \text{Smell–Pass Vulnerability} \tag{3}$$
$$\Omega_3 = (CBO/40)(1 - MS/100), \quad w_3 = +0.10 \quad \text{Coupling–Mutation Brittleness} \tag{4}$$
$$\Omega_4 = (MI/100)(1 - DD/15), \quad w_4 = -0.08 \quad \text{Maintainability Resilience} \tag{5}$$
$$\Omega_5 = (DUP/60)(FTR/30), \quad w_5 = +0.08 \quad \text{Duplication–Flakiness Chaos} \tag{6}$$

Positive hazard weights increase predicted defect probability when weaknesses compound. The negative maintainability coefficient reduces predicted risk when maintainability is high and defect density is low.

3.3 Multi-Dimensional Quality Scoring

$$\text{Quality Score} = \text{round}((1 - P_{\text{defect}}) \times 100). \tag{7}$$

$$\begin{aligned}\text{Reliability Index} = \text{round}(&\min(100, MTBF/5) \times 0.30 \\ &+ \max(0, 100 - 6.67DD) \times 0.30 + TPR \times 0.20 + MI \times 0.20).\end{aligned} \tag{8}$$

$$\begin{aligned}\text{Maintainability Index} = \text{round}(&MI \times 0.35 + (100 - 1.5DUP) \times 0.20 \\ &+ CR \times 0.15 + (100 - 2.5CBO) \times 0.15 + (100 - 2CS) \times 0.15).\end{aligned} \tag{9}$$

4 EXPERIMENTAL RESULTS AND ANALYSIS

4.1 Benchmark Evaluation on 50 Real-World Modules

The experimental corpus comprises 50 enterprise software modules (38,450 lines of code) evaluated using 10-fold cross-validation.

Table 2: Comparative Performance Across Machine Learning Models

Predictive Architecture	**Accuracy**	**Precision**	**Recall**	**F1**	**AUC**	**MSE**
Support Vector Machine (Baseline)	72.3%	70.8%	69.5%	70.1%	0.76	0.112
Random Forest (Baseline)	75.8%	73.4%	72.9%	73.1%	0.79	0.098
Neural Network (Static Only)	78.4%	76.2%	74.8%	75.5%	0.82	0.087
Gradient Boosting (Dynamic Only)	81.2%	79.5%	80.1%	79.8%	0.85	0.072
Proposed Hybrid AI Framework	**93.6%**	**92.1%**	**91.8%**	**91.9%**	**0.96**	**0.031**

4.2 Statistical Significance Analysis

A Wilcoxon signed-rank test reported a 12.4% accuracy improvement over the strongest single model ($W = 55.0$, $Z = 2.81$, $p = 0.00049 < 0.001$). Cohen's $d = 1.34$ was reported as the effect size.

4.3 Confusion Matrix and Error Distribution

Across 400 test-module evaluations, the reported results were: True Positives = 184, True Negatives = 183, False Positives = 16, and False Negatives = 17. The reported false positive rate was 8.04%, and false negative rate was 8.45%.

4.4 Feature Importance Distribution

Table 3: Primary Feature Importance Rankings

Rank	Static Metric	Gini	Dynamic Metric	Gini
1	Cyclomatic Complexity (CC)	0.189	Branch Coverage (BC)	0.198
2	Maintainability Index (MI)	0.162	Mutation Score (MS)	0.175
3	Code Smells (CS)	0.138	Defect Density (DD)	0.158
4	Coupling (CBO)	0.121	Test Pass Rate (TPR)	0.132
5	Halstead Volume (HV)	0.105	Statement Coverage (SC)	0.109

4.5 Case Study: Multi-Scenario Diagnostics

Five software engineering archetypes were evaluated across varying architectural and testing conditions. The table below follows the revised paper and omits the remedial-directive column.

Table 4: Multi-Scenario Empirical Diagnostics and Model Inferences

Scenario	Key Metrics	Static NN	Dynamic GBM	Synergy	Defect Prob.	Quality	Risk
1. All-Critical Toxic Module	CC=80; BC=5%; MI=0; MTBF=1h	0.750	0.812	+0.372	0.990	1/100	Critical
2. Superficial Coverage Trap	SC=94%; BC=32%; MS=22%	0.482	0.240	+0.222	0.564	44/100	High
3. Legacy Spaghetti Monolith	LOC=4.2k; CC=58; CBO=31	0.710	0.680	+0.128	0.821	18/100	Critical
4. Flaky Pipeline Disaster	FTR=26%; MTBF=35h; CC=8	0.180	0.520	+0.064	0.442	56/100	High
5. Clean Microservice	CC=4; BC=91%; MS=89%; MI=88	0.105	0.082	−0.057	0.069	93/100	Low

5 DISCUSSION

The reported results suggest that defect propensity cannot be modelled effectively within a single analytical silo. The reported performance (93.6% accuracy and 0.96 AUC) is attributed to combining static and dynamic signals.

Superficial coverage trap: For modules with 94% statement coverage, 32% branch coverage, and a 22% mutation score, the framework reports a 56% defect probability and classifies the module as high risk.

CI/CD feasibility: The paper reports inference latency under 5 milliseconds and model training/retraining time of 3.7 seconds, suggesting potential integration into GitHub Actions and Jenkins pull-request workflows.

6 CONCLUSION AND FUTURE WORK

This paper presents a dual-pipeline Hybrid AI Framework combining neural static analysis and gradient-boosted dynamic telemetry. An attention-weighted fusion layer and explicit cross-feature interactions aim to reduce blind spots in defect prediction. The paper reports 93.6% accuracy, 0.96 AUC-ROC, and a 56.9% reduction in MSE compared with single-paradigm baselines.

Future work: Extensions include Graph Neural Networks (GNNs) over Interprocedural Control-Flow Graphs (ICFG) and fine-tuning Code Large Language Models (LLMs) to generate patches for prioritized remediation actions.

REFERENCES

[1] A. H. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976. https://doi.org/10.1109/TSE.1976.233837

[2] M. H. Halstead, *Elements of Software Science*. New York: Elsevier North-Holland, 1977.

[3] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, June 1994. https://doi.org/10.1109/32.295895

[4] C. Catal and B. Diri, “A systematic review of software fault prediction studies,” *Expert Systems with Applications*, vol. 36, no. 4, pp. 7346–7354, May 2009. https://doi.org/10.1016/j.eswa.2008.10.027

[5] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are mutants a valid substitute for real faults in software testing?,” in *Proc. 22nd ACM SIGSOFT FSE*, Hong Kong, 2014, pp. 654–665. https://doi.org/10.1145/2635868.2635929

[6] Q. Luo, M. J. Harrold, D. Poshyvanyk, and P. McMinn, “An empirical analysis of flaky tests,” in *Proc. 22nd ACM SIGSOFT FSE*, Hong Kong, 2014, pp. 643–653. https://doi.org/10.1145/2635868.2635920

[7] M. Gligoric, A. Groce, C. Zhang, R. Sharma, M. A. Al-Ghamdi, and D. Marinov, “Comparing non-adequate test suites using coverage criteria,” in *Proc. 2015 ISSTA*, Baltimore, 2015, pp. 302–313. https://doi.org/10.1145/2771783.2771796

[8] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, “Automated parameter optimization of classification techniques for defect prediction,” *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 760–778, 2016. https://doi.org/10.1109/TSE.2016.2584050

[9] S. Wang, T. Liu, and L. Tan, “Automatically learning semantic features for defect prediction,” in *Proc. 38th ICSE*, Austin, 2016, pp. 297–308. https://doi.org/10.1145/2884781.2884804

[10] T. Menzies, J. Greenwald, and A. Frank, “Data mining static code attributes to learn defect predictors,” *IEEE Transactions on Software Engineering*, vol. 33, no. 1, pp. 2–13, 2007. https://doi.org/10.1109/TSE.2007.256941

[11] N. Fenton and M. Neil, “A critique of software defect prediction models,” *IEEE Transactions on Software Engineering*, vol. 25, no. 5, pp. 675–689, 1999. https://doi.org/10.1109/32.815326

[12] T. Hall, S. Beecham, D. Bowes, D. Gray, and K. Counsell, “A systematic literature review on fault prediction performance in software engineering,” *IEEE Transactions on Software Engineering*, vol. 38, no. 6, pp. 1276–1304, 2012. https://doi.org/10.1109/TSE.2011.103

[13] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, “Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities,” *IEEE Transactions on Software Engineering*, vol. 37, no. 6, pp. 772–787, 2011. https://doi.org/10.1109/TSE.2010.81

[14] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. https://doi.org/10.1214/aos/1013203451